

4-1회차 | 분류 ① 로지스틱 회귀

핵심 질문: "확률로 분류"한다는 건 무슨 뜻인가?

오늘의 목표

1. **분류란?**: 데이터를 보고 어느 그룹에 속하는지 라벨을 붙이는 작업
2. **Sigmoid 함수**: 모델 점수(z)를 확률(0~1)로 바꿔주는 변환기
3. **로지스틱 회귀**: Sigmoid를 이용한 확률 기반 분류기
4. **임계값(Threshold)**: 임계값에 따라 Precision/Recall이 줄다리기

예제: Titanic 생존 예측 — 예측 확률 출력 → 임계값 실험

분류를 다루는 이유

전처리와 평가지표를 배움.

데이터를 준비하는 방법과 모델 성능을 제대로 평가하는 방법을 알았을 것

이제 본격적으로 **분류 모델**을 배울 차례

분류(Classification)란?

데이터를 보고 **어느 그룹에 속하는지 라벨을 붙이는 작업**

| 분야 | 분류 예시 |

|-----|-----|

| 금융 | 사기 거래인가, 정상 거래인가? |

| 이메일 | 스팸인가, 정상 메일인가? |

| 의료 | 암 환자인가, 건강한 사람인가? |

| 이미지 | 고양이인가, 강아지인가? |

| 마케팅 | 이 고객이 이탈할 것인가? |

> **4회차 핵심 문장:**

> "분류 모델은 각각 다른 방식으로 경계를 긋는다 — 확률, 거리, 규칙."

대표적인 분류 알고리즘

알고리즘	핵심 아이디어	기준
로지스틱 회귀	점수 → 확률 → 분류	확률
KNN	가까운 이웃끼리 다수결	거리
결정 트리	질문을 반복해서 분류	규칙

Part 1. Sigmoid 함수 — 점수를 확률로 바꿔주는 변환기

왜 확률이 필요한가?

- 모델이 내부적으로 계산한 점수(z)는 $-\infty \sim +\infty$ 범위
- 하지만 우리가 원하는 건 "이 사람이 생존할 확률이 몇 %인가?"
- Sigmoid 함수가 이 점수를 **0 ~ 1** 사이의 확률로 바꿔줌

Sigmoid 공식

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

z 값	sigmoid(z)	해석
-10	≈ 0.00	거의 확실히 음성
0	0.50	반반 (애매)
+10	≈ 1.00	거의 확실히 양성

> 마치 시험 점수(z)를 합격 확률로 바꿔주는 변환기!

```
In [1]: import numpy as np
import plotly.graph_objects as go

z = np.linspace(-10, 10, 200)

sigmoid = 1 / (1 + np.exp(-z))

fig = go.Figure()
fig.add_trace(go.Scatter(x=z, y=sigmoid, mode="lines",
                        line=dict(color="purple", width=3),
                        name="Sigmoid"))

fig.add_hline(y=0.5, line_dash="dash", line_color="grey",
              annotation_text="확률 0.5 (기준선)")
fig.add_vline(x=0, line_dash="dash", line_color="grey")

fig.update_layout(title="Sigmoid Function (점수 → 확률 변환기)",
                  xaxis_title="z (모델 점수)",
                  yaxis_title="sigmoid(z) = 확률",
                  template="plotly_dark",
                  yaxis=dict(range=[-0.05, 1.05]))

fig.show()
```

Sigmoid 그래프 해석

- **z가 크면** → $\text{sigmoid} \approx 1$ → "양성(생존)일 확률이 높다" (확신)
- **z가 작으면** → $\text{sigmoid} \approx 0$ → "음성(사망)일 확률이 높다" (확신)
- **z = 0** → $\text{sigmoid} = 0.5$ → "반반, 가장 애매한 지점" (**결정 경계**)

> 핵심 1: **z = 0** ⇔ **확률 0.5** 가 기본 기준선

> 핵심 2: 경계 근처는 **민감(확률이 크게 흔들림)**, 양끝은 **둔감(0/1에 붙음)** → 헷갈림/확신이 생김

Part 2. Titanic 데이터 전처리

Titanic 데이터셋

- 실제 타이타닉호 생존자 데이터
- 목표: 승객의 나이, 성별, 선실 등급 등을 보고 **생존 여부(0=사망, 1=생존)** 예측
- 주요 컬럼:
 - Pclass : 선실 등급 (1=일등석, 2=이등석, 3=삼등석)
 - Sex : 성별
 - Age : 나이
 - Fare : 운임 요금
 - **Target(정답):** Survived (1=생존, 0=사망)

> 3회차에서 배운 전처리(인코딩, 결측치 처리)를 활용

```
In [2]: import pandas as pd
        titanic_df = pd.read_csv('train.csv')
        titanic_df.head()
```

Out[2]:

	PassengerId	Survived	Pclass	Name	Sex	Age	Survived
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0

```
In [3]: import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

titanic_df = pd.read_csv("train.csv")

y_titanic_df = titanic_df["Survived"]
X_titanic_df = titanic_df.drop("Survived", axis=1).copy()

X_titanic_df["Cabin1"] =
X_titanic_df["Cabin"].fillna("N").astype(str).str[:1]

X_titanic_df = X_titanic_df.drop(["PassengerId", "Name",
                                "Ticket", "Cabin"], axis=1)

print(f"데이터 크기: {X_titanic_df.shape}")
print(f"생존 비율:\n{y_titanic_df.value_counts(normalize=True)}")
display(X_titanic_df.head())

X_train, X_test, y_train, y_test = train_test_split(
    X_titanic_df, y_titanic_df,
    test_size=0.2, stratify=y_titanic_df, random_state=42
)

numeric_features = ["Pclass", "Age", "SibSp", "Parch", "Fare"]
categorical_features = ["Sex", "Embarked", "Cabin1"]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ],
    remainder="drop"
)

lr = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("model", LogisticRegression(max_iter=5000,
                                random_state=42)),
])
```

데이터 크기: (891, 8)

생존 비율:

Survived

0 0.616162

1 0.383838

Name: proportion, dtype: float64

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	male	22.0	1	0	7.2500	S
1	1	female	38.0	1	0	71.2833	C
2	3	female	26.0	0	0	7.9250	S
3	1	female	35.0	1	0	53.1000	S
4	3	male	35.0	0	0	8.0500	S

Part 3. 로지스틱 회귀로 Titanic 생존 예측

로지스틱 회귀란?

- "회귀"라는 이름이 붙어있지만 **분류 모델**임
- 내부적으로 **선형 회귀처럼 점수(z)**를 계산한 뒤
- **Sigmoid 함수로 확률(0~1)로 변환**
- 확률이 임계값(기본 0.5) 이상이면 → 양성(1)

、

입력 데이터 → 점수(z) 계산 → Sigmoid → 확률 → 임계값 비교 → 예측

、

| 단계 | 내용 | 예시 |

|-----|-----|-----|

| 1 | 점수 계산 | $z = 2.3$ |

| 2 | Sigmoid 변환 | $\sigma(2.3) = 0.909$ |

| 3 | 임계값 비교 | $0.909 \geq 0.5 \rightarrow$ 양성(생존) |

로지스틱 회귀의 본질

로지스틱 회귀를 한 마디로 말하면:

> **입력을 점수(z)로 만들고, 그 점수를 sigmoid로 눌러 확률로 만든 뒤, 임계값으로 분류하는 모델**

> (핵심: 로지스틱은 0/1을 바로 찍지 않고, 확률을 먼저 만든다)

경계선(Decision Boundary)이란?

![로지스틱 회귀는 데이터를 가르는 직선을 긋는 모델](스크린샷%202026-02-16%20오후%202.13.03.png)

로지스틱 회귀는 두 클래스를 나누는 **직선(경계선)**을 찾음.

| 위치 | 확률 | 의미 |

|-----|-----|-----|

| 직선 위 (경계선) | 0.5 | 반반, 애매함 |

| 직선에서 **양성** 쪽으로 멀어짐 | → 1.0 | 양성일 가능성 높음 |

| 직선에서 **음성** 쪽으로 멀어짐 | → 0.0 | 음성일 가능성 높음 |

> **주의:** 직선은 "예측값"이 아니라, **두 클래스를 나누는 기준선임!**

> 경계선에 가까울수록 확률이 0.5에 가깝고 (모델이 헛갈림),

> 멀어질수록 확률이 0이나 1에 가깝습니다 (모델이 확신함).

![점수(z)가 클수록 확률이 0이나 1로 치우쳐 '확신'이 커집니다](스크린샷%202026-02-16%20오후%202.13.29.png)

그 직선이 "잘 나눈다"는 걸 어떻게 아는가?

1회차에서 배운 **손실함수**를 기억!

로지스틱 회귀는 **Cross Entropy Loss**를 사용함:

| 상황 | 벌점 |

|-----|-----|

| 정답 클래스의 확률이 **1에 가까우면** | 벌점 아주 작음 |

| 정답 클래스의 확률이 **0.5면** | 벌점 상당함 |

| 정답 클래스의 확률이 **0에 가까우면** | 벌점 엄청 큼! |

> 1회차에서 본 "메인 재료 빠지면 극대노 미식가" = Cross Entropy!

학습 과정 (5단계)

| 단계 | 모델이 하는 일 |

|-----|-----|

| 1 | 직선을 **아무렇게나** 긋는다 || 2 | 각 데이터의 **확률**을 계산한다 || 3 | 정답과 비교해서 **벌점(Cross Entropy)**을 받는다 || 4 | 벌점이 줄어드는 방향으로 **직선을 조금 움직인다** || 5 | 이걸 **수천 번 반복** → 손실이 최소가 되는 직선에 도착! |

![벌점을 줄이기 위해 끊임없이 선을 움직입니다](스크린샷%202026-02-16%20오후%202.14.23.png)

> **핵심**: 직접 "잘 나누는지" 판단하는 게 아님!

> **확률 예측 오차(Cross Entropy)**를 최소화하다 보니,

> 결과적으로 잘 나누는 선이 생기는 것!

로지스틱 회귀가 하는 일 (한 문장 정리)

> 정답 클래스는 확률이 1에 가깝게,

> 반대 클래스는 확률이 0에 가깝게

> 만드는 직선을 찾아가는 것

로지스틱 회귀의 가정 — "세상이 직선이 아니면?"

로지스틱 회귀는 "**직선으로 나눌 수 있는 세계**"를 가정

| 상황 | 결과 |

|-----|-----|

| 직선으로 나눌 수 있는 데이터 | 잘 작동함 |

| 직선으로 나눌 수 **없는** 데이터 | 한계가 있음 |

> 세상이 직선이 아니면?

> → **모델을 바꾸거나** (결정 트리, SVM 등)

> → **입력을 바꿔야** 합니다 (다항식 특성 추가 등)

> 이걸 다음 시간에 결정 트리를 배우면서 비교해볼 예정!

```
In [4]: lr.fit(X_train, y_train)

y_proba = lr.predict_proba(X_test)[: , 1]

pd.DataFrame({"예측 확률 (생존)": y_proba[:10].round(4)})
```

Out[4]:

	예측 확률 (생존)
0	0.0669
1	0.0467
2	0.1610
3	0.0345
4	0.6714
5	0.4439
6	0.7027
7	0.2944
8	0.3258
9	0.1268

predict vs predict_proba

메서드	반환값	예시
predict()	0 또는 1	[0, 1, 0, 1]
predict_proba()	각 클래스의 확률	[[0.92, 0.08], [0.27, 0.73]]

- predict_proba()[:, 0] = 사망 확률
- predict_proba()[:, 1] = 생존 확률

> predict() 는 내부적으로 predict_proba() → 임계값 0.5 비교를 자동으로 해주는 것!

```
In [5]: from sklearn.metrics import accuracy_score

# 기본 예측 (임계값 0.5)
y_pred = lr.predict(X_test)
print(f"기본 Accuracy (threshold=0.5): {accuracy_score(y_test,
y_pred):.3f}")
```

기본 Accuracy (threshold=0.5): 0.816

Part 4. 임계값(Threshold)에 따른 Precision/Recall 줄다리기

복습: Precision과 Recall

| 지표 | 공식 | 의미 |

|-----|-----|-----|

| **Precision** | $TP / (TP + FP)$ | "양성이라고 했는데, 진짜 양성이 맞나?" |

| **Recall** | $TP / (TP + FN)$ | "진짜 양성 중에서 얼마나 찾았나?" |

임계값을 바꾸면?

| 임계값 | Recall | Precision | 의미 |

|-----|-----|-----|-----|

| 낮춤 (0.3) | ↑ | ↓ | 더 많이 양성으로 잡음 (누락↓, 오탐↑) |

| 높임 (0.7) | ↓ | ↑ | 확실할 때만 양성 (오탐↓, 누락↑) |

> Precision과 Recall은 줄다리기 관계!

> 한쪽을 올리면 다른 쪽이 내려갑니다.

> 그래서 이 문제(타이타닉)에서 FP와 FN 중 무엇이 더 비싼가?

```

In [6]: from sklearn.metrics import precision_score, recall_score,
confusion_matrix
import pandas as pd

for thr in [0.3, 0.5, 0.7]:
    y_hat = (y_proba >= thr).astype(int)

    print(f"Threshold={thr}")
    print(f" Precision: {precision_score(y_test, y_hat,
zero_division=0):.4f}")
    print(f" Recall   : {recall_score(y_test, y_hat,
zero_division=0):.4f}")

    cm = confusion_matrix(y_test, y_hat)
    cm_df = pd.DataFrame(cm, index=["Actual 0", "Actual 1"],
columns=["Pred 0", "Pred 1"])
    cm_df
    print()

```

Threshold=0.3
Precision: 0.6506
Recall : 0.7826

	Pred 0 (사망)	Pred 1 (생존)
Actual 0 (사망)	81	29
Actual 1 (생존)	15	54

FP(오탐): 29 FN(누락): 15 TP: 54 TN: 81

Threshold=0.5
Precision: 0.8103
Recall : 0.6812

	Pred 0 (사망)	Pred 1 (생존)
Actual 0 (사망)	99	11
Actual 1 (생존)	22	47

FP(오탐): 11 FN(누락): 22 TP: 47 TN: 99

Threshold=0.7
Precision: 0.8974
Recall : 0.5072

	Pred 0 (사망)	Pred 1 (생존)
Actual 0 (사망)	106	4
Actual 1 (생존)	34	35

FP(오탐): 4 FN(누락): 34 TP: 35 TN: 106

- > - 임계값 0.3 → Recall 높음 (많이 잡음), Precision 낮음 (오탐 많음)
- > - 임계값 0.7 → Precision 높음 (확실할 때만), Recall 낮음 (놓침 많음)
- > - 임계값 0.5 → 중간 균형

```
In [7]: from sklearn.metrics import f1_score
import plotly.graph_objects as go

thr_list = np.round(np.arange(0.1, 1.0, 0.1), 2)
rows = []
for thr in thr_list:
    y_hat = (y_proba >= thr).astype(int)
    rows.append([
        thr,
        precision_score(y_test, y_hat, zero_division=0),
        recall_score(y_test, y_hat, zero_division=0),
        f1_score(y_test, y_hat, zero_division=0)
    ])

thr_df = pd.DataFrame(rows, columns=["Threshold", "Precision",
"Recall", "F1"])
thr_df
```

Out[7]:

	Threshold	Precision	Recall	F1
0	0.1	0.456522	0.913043	0.608696
1	0.2	0.600000	0.826087	0.695122
2	0.3	0.650602	0.782609	0.710526
3	0.4	0.698630	0.739130	0.718310
4	0.5	0.810345	0.681159	0.740157
5	0.6	0.851064	0.579710	0.689655
6	0.7	0.897436	0.507246	0.648148
7	0.8	0.954545	0.304348	0.461538
8	0.9	0.916667	0.159420	0.271605

```
In [8]: fig = go.Figure()
fig.add_trace(go.Scatter(x=thr_df["Threshold"],
                        y=thr_df["Precision"],
                        mode="lines+markers", name="Precision",
                        line=dict(dash="dash", color="cyan"))))
fig.add_trace(go.Scatter(x=thr_df["Threshold"],
                        y=thr_df["Recall"],
                        mode="lines+markers", name="Recall",
                        line=dict(color="magenta"))))
fig.add_trace(go.Scatter(x=thr_df["Threshold"], y=thr_df["F1"],
                        mode="lines+markers", name="F1",
                        line=dict(color="yellow"))))

fig.update_layout(title="임계값 변화에 따른 Precision / Recall / F1",
                  xaxis_title="Threshold",
                  yaxis_title="Score",
                  template="plotly_dark",
                  yaxis=dict(range=[0, 1]))

fig.show()
```

임계값 선택 가이드

목표가 무엇인가?

```
├── Recall이 중요 (암 진단, 사기 탐지)
|   ├── 임계값 낮추기 (0.3~0.4)
├── Precision이 중요 (스팸 필터, 추천)
|   ├── 임계값 높이기 (0.6~0.7)
├── 균형이 중요
└── F1이 최대인 임계값 선택
```

상황	더 중요한 지표	임계값 방향	이유
암 진단	Recall	↓ 낮춤	환자를 놓치면 생명 위험
스팸 필터	Precision	↑ 높임	정상 메일을 스팸으로 분류하면 안 됨
금융 사기	Recall	↓ 낮춤	사기를 놓치면 큰 손실
추천 시스템	Precision	↑ 높임	잘못된 추천은 사용자 이탈

> 임계값은 "정답"이 없음 업무 목표(정책)에 따라 선택해야함!

Part 5. 확률 분포 시각화

모델이 출력한 **확률 분포**를 보면 임계값 선택의 감이 옴

```
In [9]: import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Histogram(
    x=y_proba[y_test == 0], name="사망 (실제=0)",
    opacity=0.6, marker_color="cyan",
    xbins=dict(start=0, end=1, size=0.05)
))
fig.add_trace(go.Histogram(
    x=y_proba[y_test == 1], name="생존 (실제=1)",
    opacity=0.6, marker_color="magenta",
    xbins=dict(start=0, end=1, size=0.05)
))

fig.add_vline(x=0.5, line_dash="dash", line_color="yellow",
              annotation_text="threshold=0.5")

fig.update_layout(
    title="실제 라벨별 예측 확률 분포",
    xaxis_title="예측 확률 (생존)",
    yaxis_title="빈도",
    barmode="overlay",
    template="plotly_dark"
)
fig.show()
```

확률 분포 해석

- **사망(파란색)** 분포가 왼쪽(낮은 확률)에 몰려 있으면 → 모델이 사망자를 잘 구분
- **생존(분홍색)** 분포가 오른쪽(높은 확률)에 몰려 있으면 → 모델이 생존자를 잘 구분
- 두 분포가 **겹치는 구간** = 모델이 헛갈리는 구간 → 임계값 조정이 필요한 곳

> 좋은 모델일수록 두 분포가 **확실하게 분리**됨

모델 성능에 따른 확률 분포 비교

좋은 모델일수록 두 분포가 확실하게 분리됨.

피처를 점점 늘려가며 모델 성능을 높여보고, 확률 분포가 어떻게 변하는지 관찰해보기

| 모델 | 사용 피처 | 예상 |

|-----|-----|-----|

| ① 약한 모델 | Pclass만 | 분포가 거의 겹침 |

| ② 중간 모델 | Pclass + Sex | 어느 정도 분리 |

| ③ 현재 모델 | 전체 피처 | 확실하게 분리 |

```

In
[12]: from plotly.subplots import make_subplots
      from sklearn.metrics import roc_auc_score

      from sklearn.compose import ColumnTransformer
      from sklearn.pipeline import Pipeline
      from sklearn.impute import SimpleImputer
      from sklearn.preprocessing import OneHotEncoder, StandardScaler
      from sklearn.linear_model import LogisticRegression

      all_numeric = {"Pclass", "Age", "SibSp", "Parch", "Fare"}
      all_categorical = {"Sex", "Embarked", "Cabin1"}

      def make_lr_pipeline(selected_features):
          selected_features = list(selected_features)
          num_feats = [f for f in selected_features if f in
all_numeric]
          cat_feats = [f for f in selected_features if f in
all_categorical]

          numeric_transformer = Pipeline(steps=[
              ("imputer", SimpleImputer(strategy="median")),
              ("scaler", StandardScaler()),
          ])

          categorical_transformer = Pipeline(steps=[
              ("imputer", SimpleImputer(strategy="most_frequent")),
              ("onehot", OneHotEncoder(handle_unknown="ignore")),
          ])

          preprocessor = ColumnTransformer(
              transformers=[
                  ("num", numeric_transformer, num_feats),
                  ("cat", categorical_transformer, cat_feats),
              ],
              remainder="drop",
          )

          return Pipeline(steps=[
              ("preprocess", preprocessor),
              ("model", LogisticRegression(max_iter=5000,
random_state=42)),
          ])

      configs = [
          ("Pclass만", ["Pclass"]),
          ("Pclass + Sex", ["Pclass", "Sex"]),
          ("전체 피쳐(전처리 한거임)", X_train.columns.tolist()),
      ]

      titles, probas = [], []
      for name, feats in configs:
          m = make_lr_pipeline(feats)
          m.fit(X_train, y_train)
          p = m.predict_proba(X_test)[: , 1]
          auc = roc_auc_score(y_test, p)
          titles.append(f"{name} (AUC={auc:.3f})")

```

```

probas.append(p)

fig = make_subplots(rows=3, cols=1, subplot_titles=titles)

for i, p in enumerate(probas, 1):
    fig.add_trace(go.Histogram(
        x=p[y_test == 0], name="사망 (0)",
        opacity=0.6, marker_color="cyan",
        xbins=dict(start=0, end=1, size=0.05),
        legendgroup="died", showlegend=(i == 1)
    ), row=i, col=1)

    fig.add_trace(go.Histogram(
        x=p[y_test == 1], name="생존 (1)",
        opacity=0.6, marker_color="magenta",
        xbins=dict(start=0, end=1, size=0.05),
        legendgroup="survived", showlegend=(i == 1)
    ), row=i, col=1)

fig.update_layout(
    height=900,
    barmode="overlay", template="plotly_dark",
    title="모델이 좋아질수록 확률 분포가 분리!",
)

for i in range(1, 4):
    fig.update_xaxes(range=[0, 1], row=i, col=1)

fig.show()

```

해석

모델	분포 모양	AUC	의미
Pclass만	거의 겹침	낮음	구분 능력 부족
Pclass + Sex	어느 정도 분리	중간	성별 추가로 크게 개선
전체 피쳐	확실히 분리	높음	좋은 분류 모델

> **핵심**: 좋은 모델 = 사망자에게 **낮은 확률**, 생존자에게 **높은 확률**을 확신 있게 부여
 > → 두 분포가 **멀리 떨어질수록** 분류 성능이 좋다!
 > → 이것이 바로 **AUC가 높다**는 의미!

왜 Sex를 추가하면 분포가 확 벌어질까?

Pclass만 썼을 때의 한계

Pclass만을 보면:

- 사망과 생존의 확률 분포가 **많이 겹쳐 있음**
- AUC가 0.5(랜덤)는 아니니까 Pclass는 분명 영향이 있음
- 하지만 3등석 중에도 생존자가 있고, 1등석 중에도 사망자가 있음
- → 확률이 **0.3~0.6** 근처로 애매하게 나옴

> Pclass만 쓰면 모델이 이렇게 말하는 셈:

> "3등석이면 좀 위험하긴 한데... 100%는 아니야"

"분포가 겹친다" = 어떤 구간에서 사망도 많고 생존도 많다 = threshold 0.5에서 헛갈린다!

Sex를 추가하면 왜 급변하는가?

타이타닉에는 거의 규칙처럼 작동한 게 있었음:

> "**Women and children first**" (여성과 아이 먼저)

![현실의 규칙이 수식의 가중치(Weight)로 변환되었습니다](스크린샷%202026-02-16%20오후%202.12.24.png)

그룹	생존률
여성	≈ **70% 이상**
남성	≈ **20% 이하**

이건 엄청 강한 신호!

모델 관점에서 보면: $z = w_1 \cdot \text{Pclass} + w_2 \cdot \text{Sex} + b$

Sex 값	z 방향	sigmoid 후 확률
여성	z가 크게 양수	0.7 ~ 0.9
남성	z가 크게 음수	0.05 ~ 0.2

→ 양쪽 끝으로 확 갈라짐!

AUC와의 연결

AUC = "생존자가 사망자보다 더 높은 점수를 받을 확률"

Sex가 들어가면서:

- 거의 모든 여성(생존 많음) > 거의 모든 남성(사망 많음)
- → 순위 정렬이 훨씬 정확해짐 → **AUC 급상승**

> **한 줄 요약:** Sex가 추가되면서 모델이 생존과 사망을

> 거의 **양쪽 극단으로 밀어버릴 수 있게** 되었기 때문!

그렇다면 Sex 하나만 써도 AUC가 높을까?

Sex가 그만큼 강한 신호라면, **Sex 하나만으로도** AUC가 꽤 높을 수 있지 않을까?

아래 코드로 직접 확인해보기!

```
In [11]: from sklearn.metrics import roc_auc_score, accuracy_score
import pandas as pd

configs_ext = [
    ("Pclass만", ["Pclass"]),
    ("Sex만", ["Sex"]),
    ("Pclass + Sex", ["Pclass", "Sex"]),
    ("전체 피쳐", X_train.columns.tolist()),
]

rows = []
for name, feats in configs_ext:
    m = make_lr_pipeline(feats)
    m.fit(X_train, y_train)
    p = m.predict_proba(X_test)[: , 1]
    auc = roc_auc_score(y_test, p)
    acc = accuracy_score(y_test, (p >= 0.5).astype(int))
    rows.append([name, len(feats), f"{auc:.3f}", f"{acc:.3f}"])

pd.DataFrame(rows, columns=["모델", "피쳐 수", "AUC", "Accuracy"])
```

Out[11]:

	모델	피쳐 수	AUC	Accuracy
0	Pclass만	1	0.668	0.642
1	Sex만	1	0.753	0.777
2	Pclass + Sex	2	0.822	0.777
3	전체 피쳐	8	0.841	0.816

해석

- **Sex 하나만으로도** Pclass보다 높은 AUC를 보임
- Sex + Pclass 두 개만 합쳐도 전체 피처에 근접하는 성능
- 나머지 피처(Age, Fare 등)는 **예외 케이스**를 더 잘 분리해주는 역할

| 피처 | 역할 |

|-----|-----|

| **Sex** | 가장 강한 신호 — 생존/사망의 **큰 틀**을 결정 |

| **Pclass** | 보조 신호 — 같은 성별 안에서 **추가 구분** |

| **나머지** | 미세 조정 — **예외 케이스** 처리 |

- > **교훈**: 피처를 많이 넣는다고 무조건 좋은 게 아니라,
- > **강한 신호 1~2개**가 성능의 대부분을 결정할 수 있다!

오늘의 정리

| 개념 | 핵심 | 코드 |

|-----|-----|-----|

| 분류 | 데이터에 라벨 붙이기 | — |

| Sigmoid | 점수(z) → 확률(0~1) 변환기 | $1 / (1 + \exp(-z))$ |

| 로지스틱 회귀 | 확률 기반 분류기 | LogisticRegression() |

| predict_proba | 각 클래스의 확률 출력 | lr.predict_proba(X)[: ,1] |

| 임계값 | 확률 → 예측 변환 기준 | (y_proba >= thr).astype(int) |

| Precision/Recall | 임계값에 따라 줄다리기 | 정책에 맞게 조정 |

> **기억할 흐름**:

> 로지스틱 회귀 = **Sigmoid 함수** → 점수(z)를 **확률로 변환** → **임계값에 따라 분류** → **정책에 맞게 임계값을 조정**

다음: 4-2 KNN

- **KNN**: 거리 기반 다수결 분류기
- **K 값**에 따른 경계 변화 (과적합 ↔ 과소적합)
- 스케일링이 왜 필수인지
- **Pipeline + GridSearchCV**로 최적 K 탐색
- **Decision Boundary** 시각화

- > 로지스틱 회귀는 **확률**로 분류했다면,
- > KNN은 **가까운 이웃들의 다수결**로 분류!

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).